

UNIVERSITÀ DI PISA
DIPARTIMENTO DI INFORMATICA
Data Science and Business Informatics

Project 29

Multi-class SVM with a Lagrangian
Dual and Bundle Methods

Professor:
Prof. Antonio Frangioni

Students:
Tolgonai Nasipbek kyzy
Hamse Hassan Adnan

ANNO ACCADEMICO 2025/2026

Contents

1	Introduction	3
1.1	Description of the task	3
2	SVM Problem Formulation	4
2.1	Soft margin formulation	4
2.2	Dual formulation	5
2.3	Primal and dual variables	7
2.4	Kernel trick	7
3	The Lagrangian Dual	8
3.1	What a Lagrangian relaxation is for	8
3.2	Which constraints to dualise	9
3.3	Putting w back	10
3.4	The relaxed problem is a continuous knapsack	11
3.5	Properties of the dual function	12
3.6	What this dual actually is	12
3.7	Relation to the direct nonsmooth formulation	13
4	The Bundle Method	14
4.1	The cutting-plane model and the bundle	14
4.2	The structured master problem	15
4.3	Primal recovery and Dantzig–Wolfe	16
4.4	Kernelised form	16
4.5	Cost, and how many cuts are needed	17
5	The General-Purpose Solver	18
6	Multiclass SVM	19
6.1	One-vs-Rest approach	19
6.2	Does solving one problem help the others?	19

7 Experiments	21
7.1 Correctness	21
7.2 Against the direct nonsmooth baseline	22
7.3 Bundle size and free support vectors	22
7.4 One-vs-Rest transfer	22
A The shared-slack-budget model	24
A.1 The model	24
A.2 Its dual	24
A.3 The defect, and its repair	25
A.4 Why it is not the main line	25

Chapter 1

Introduction

1.1 Description of the task

This report explores two different optimization algorithms applied to a machine learning model.

The ML model (M) is a Support Vector Machine-type approach for multi-class classification, with a choice of kernels, using the standard One-vs-Rest approach.

The first optimization algorithm (A1) is a dual approach applied to the dual formulation of the SVM. We reintroduce the weight vector w as an explicit variable, so that the representer identity $w = \sum_i \alpha_i y_i \phi(x_i)$ becomes a constraint of the problem, and we dualise *that* identity. The resulting Lagrangian relaxation is a linear program over a box with a single equality constraint — a continuous knapsack, solved exactly by sorting in $O(m \log m)$ — so the dual function is finite everywhere, genuinely non-differentiable, and defined on a space of dimension n for a linear kernel or m in general. It is maximised by a proximal bundle method, whose master problem is solved by an off-the-shelf QP solver, as the project statement allows.

The second optimization algorithm (A2) is a general-purpose convex QP solver applied to the standard SVM dual. No off-the-shelf solver is used elsewhere, save for the master problem of the bundle method.

Specific discussion is provided in Chapter 6 about if and how solving one training problem in the One-vs-Rest approach may provide useful solutions that help in solving the others.

Chapter 2

SVM Problem Formulation

The goal of this chapter is to describe the Support Vector Machine and to derive the Lagrangian dual formulation that defines the optimization task as a quadratic programming problem. The material is standard; it is included because Chapter 3 departs from it at one specific point, the stationarity condition (2.2), and that departure is easier to follow with the usual derivation in front of the reader.

2.1 Soft margin formulation

Let $\{(x_i, y_i)\}_{i=1}^m$ with $x_i \in \mathbb{R}^n$ and $y_i \in \{-1, +1\}$, and let $\phi : \mathbb{R}^n \rightarrow \mathcal{H}$ be a feature map into a Hilbert space with reproducing kernel $\kappa(x, x') = \langle \phi(x), \phi(x') \rangle$. Write $\phi_i := \phi(x_i)$.

A separating hyperplane $H(w, b) = \{x : \langle w, \phi(x) \rangle + b = 0\}$ classifies correctly with margin at least one when $y_i(\langle w, \phi_i \rangle + b) \geq 1$ for all i , and the geometric margin is then $1/\|w\|$, so maximising the margin means minimising $\|w\|$. Since such a hyperplane need not exist, each sample is given a slack $\xi_i \geq 0$ by which it may fall short of its own margin, penalised linearly with weight $C > 0$. This gives the soft-margin (C -)SVM:

$$\begin{aligned} \min_{w \in \mathcal{H}, b \in \mathbb{R}, \xi \in \mathbb{R}^m} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i \\ \text{s.t.} \quad & y_i(\langle w, \phi_i \rangle + b) \geq 1 - \xi_i, \quad i = 1, \dots, m, \\ & \xi_i \geq 0, \quad i = 1, \dots, m. \end{aligned} \tag{P}$$

Here $\|w\|$ measures model complexity and C weighs the empirical loss against generalisation. Problem (P) is a convex QP, strictly convex in w but only convex in (b, ξ) .

Eliminating ξ gives the equivalent unconstrained, non-smooth form

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C \sum_{i=1}^m [1 - y_i(\langle w, \phi_i \rangle + b)]^+, \quad [t]^+ := \max\{0, t\}, \quad (2.1)$$

which is the form in which the SVM appears in the course material as the motivating example for nondifferentiable optimization. We shall meet it again, unexpectedly, in Section 3.6.

2.2 Dual formulation

Attach $\alpha_i \geq 0$ to the margin constraints and $\mu_i \geq 0$ to $\xi_i \geq 0$:

$$L(w, b, \xi; \alpha, \mu) = \frac{1}{2}\|w\|^2 + C\mathbf{1}^\top \xi + \sum_{i=1}^m \alpha_i (1 - \xi_i - y_i(\langle w, \phi_i \rangle + b)) - \mu^\top \xi.$$

Stationarity in (w, b, ξ) gives the three conditions

$$w = \sum_{i=1}^m \alpha_i y_i \phi_i, \quad \sum_{i=1}^m \alpha_i y_i = 0, \quad \alpha_i + \mu_i = C. \quad (2.2)$$

The first of these deserves a name and a moment's attention, because Chapter 3 dualises precisely it. It says that at the optimum w is a linear combination of the training points, with coefficients $\alpha_i y_i$; it is the *representer identity*, the same statement as the representer theorem of kernel methods, and it is what makes kernels usable at all: however large the dimension of $\mathcal{H}$, the optimal w is described by m numbers.

It is also, as we are about to see, where Q comes from. Substituting it into $\frac{1}{2}\|w\|^2$ turns a squared norm into $\frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \kappa(x_i, x_j) = \frac{1}{2} \alpha^\top Q \alpha$: before the substitution there is no Q anywhere in the problem, and after it there is a dense $m \times m$ matrix. The dense Hessian of (D) is therefore not intrinsic to the SVM; it is the price of eliminating w .

Substituting the three conditions back into L makes every term collapse.

Writing $K_{ij} = \kappa(x_i, x_j)$, $Y = \text{diag}(y)$ and $Q = YKY$:

$$\begin{aligned} \frac{1}{2}\|w\|^2 &= \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K_{ij} = \frac{1}{2} \alpha^\top Q \alpha, \\ - \sum_i \alpha_i y_i \langle w, \phi_i \rangle &= - \left\langle w, \sum_i \alpha_i y_i \phi_i \right\rangle = -\|w\|^2 = -\alpha^\top Q \alpha, \\ -b \sum_i \alpha_i y_i &= 0 \quad (\text{by the } b \text{ condition}), \\ \sum_i \xi_i (C - \alpha_i - \mu_i) &= 0 \quad (\text{by the } \xi \text{ condition}), \end{aligned}$$

leaving $\mathbf{1}^\top \alpha - \frac{1}{2} \alpha^\top Q \alpha$, to be maximised. Two remarks on how the constraints of the dual arise, because both are easy to state loosely:

- $\mu_i \geq 0$ and $\mu_i = C - \alpha_i$ together give $\alpha_i \leq C$, which with $\alpha_i \geq 0$ is the box $0 \leq \alpha \leq C\mathbf{1}$;
- b enters L only through the term $-b \sum_i \alpha_i y_i$, and b is *free*, so $\inf_b L = -\infty$ unless $\sum_i \alpha_i y_i = 0$. The b -stationarity condition therefore does not disappear when it is used to cancel that term: it survives as the constraint $y^\top \alpha = 0$.

Writing the result as a minimisation of the negative, the dual of (P) is the convex QP

$$\min_{\alpha \in \mathbb{R}^m} \quad \frac{1}{2} \alpha^\top Q \alpha - \mathbf{1}^\top \alpha \quad \text{s.t.} \quad \alpha \in D := \{ \alpha : y^\top \alpha = 0, \ 0 \leq \alpha \leq C\mathbf{1} \}. \quad (\text{D})$$

D is a non-empty ($0 \in D$) compact polytope and $Q \succeq 0$, so (D) always has a solution. Slater's condition holds for (P), hence there is no duality gap and the KKT conditions

$$\begin{aligned} \alpha_i = 0 &\implies y_i f(x_i) \geq 1, \\ 0 < \alpha_i < C &\implies y_i f(x_i) = 1, \\ \alpha_i = C &\implies y_i f(x_i) \leq 1, \end{aligned} \quad (2.3)$$

characterise optimality, where $f(x) = \sum_i \alpha_i y_i \kappa(x_i, x) + b$. Samples with $\alpha_i > 0$ are the *support vectors*; those with $0 < \alpha_i < C$ are the *free* support vectors. Their number will turn out to control the cost of (A1); see Section 4.5.

2.3 Primal and dual variables

Given an optimal α^* of (D), the primal variables come back as

$$w^* = \sum_i \alpha_i^* y_i \phi_i, \quad \xi_i^* = [1 - y_i(\langle w^*, \phi_i \rangle + b^*)]^+,$$

the first directly from the representer identity and the second from primal feasibility of the margin constraint together with complementarity on $\xi_i \geq 0$ (note the factor y_i inside: ξ_i measures the violation of *the sample's own* margin, not of the hyperplane equation).

The bias b^* deserves more care. The textbook recipe reads b^* off any free support vector via (2.3) and averages over them for stability; but the average is only as good as the set of indices identified as free, so a solver that is merely close to optimal produces a b that degrades the measured primal–dual gap. The better observation is that b enters the Lagrangian only through $-b \sum_i \alpha_i y_i$, so b is *the multiplier of the equality constraint* $y^\top \alpha = 0$. Any algorithm that produces that multiplier explicitly can use it directly, with no support-vector bookkeeping at all. For an interior-point method it is one of the iterates; in our case the knapsack of Section 3.4 supplies it at every oracle call, as (3.4) proves by LP duality.

2.4 Kernel trick

Only inner products appear in (D), so ϕ need never be formed. The kernels used in this report are

- linear: $\kappa(x, x') = x^\top x'$;
- polynomial: $\kappa(x, x') = (\gamma x^\top x' + c_0)^d$, $\gamma > 0$;
- Gaussian (RBF): $\kappa(x, x') = \exp(-\gamma \|x - x'\|^2)$, $\gamma > 0$.

Unlike the linear case, for a nonlinear kernel $\mathcal{H}$ may be infinite-dimensional; Section 4.4 explains how the dual variable of (A1) is represented in that case.

Chapter 3

The Lagrangian Dual

3.1 What a Lagrangian relaxation is for

Before choosing what to dualise it is worth recalling what the choice is supposed to buy. Given a problem $\min\{f(x) : Ax \leq b, x \in X\}$, relaxing the constraints $Ax \leq b$ with multipliers λ removes them from the feasible set and charges for their violation in the objective, leaving

$$\psi(\lambda) = \min\{f(x) + \lambda(b - Ax) : x \in X\}.$$

Note that this defines not one problem but a *family* of them, one for each λ . We call the minimisation on the right *the relaxed problem*.

Solving it for one value of λ is what is meant, here and throughout, by a call to the *oracle*: a routine that is handed a point λ and returns the value $\psi(\lambda)$ together with one supergradient of ψ there. The name is standard and the idea behind it is that the algorithm neither knows nor needs to know how the answer is produced — ψ has no closed form, cannot be differentiated symbolically and cannot be inspected, so probing it point by point is the only thing available. Everything in Chapter 4 is built around that restriction, and the oracle call is consequently the unit in which we measure work: for the relaxation of this chapter one call costs a single kernel product and a sort, and the iteration counts reported in Chapter 7 are counts of oracle calls. Three things follow, and they hold whatever f and X look like:

- $\psi(\lambda) \leq v(P)$ for every λ , so *every* oracle call returns a valid lower bound on the optimal value;
- if a feasible point can be recovered from $x(\lambda)$, one also has a valid upper bound, and hence a readily available estimate of the gap at every iteration;

- ψ is concave and its supergradient is the constraint violation $b - Ax(\lambda)$, so the dual is a well-behaved problem even when the original is not.

The price is that one now has to search over λ . Whether the trade is worth making therefore depends entirely on *what is left in the relaxed problem*: the constraints to dualise are the *complicating* ones, and the ones to keep are those that make the residual problem easy. Section 3.2 shows that for the SVM dual the obvious candidates are exactly the wrong ones, and Section 3.4 that one non-obvious candidate is exactly the right one.

3.2 Which constraints to dualise

For a Lagrangian dual approach to be worth building, the relaxation must satisfy:

- (R1) the relaxed problem must be *substantially easier* than the original — ideally closed-form or combinatorial;
- (R2) the dual must be *multi-dimensional*, otherwise a one-dimensional search solves it and bundle machinery is pointless;
- (R3) the dual should be *genuinely non-differentiable*, otherwise a smooth method is preferable.

The obvious candidate is the single equality $y^\top \alpha = 0$. Attaching to it a free multiplier $\theta \in \mathbb{R}$ and keeping the box as the easy constraint gives the dual function

$$\psi(\theta) = \min \left\{ \frac{1}{2} \alpha^\top Q \alpha - \mathbf{1}^\top \alpha + \theta y^\top \alpha : 0 \leq \alpha \leq C \mathbf{1} \right\}. \quad (3.1)$$

Since $y^\top \alpha = 0$ is what remained of stationarity in b , dropping it removes the bias from the model: (3.1) is the bias-free SVM with a linear term $\theta y^\top \alpha$ pricing the class imbalance, and choosing θ amounts to choosing how hard to push the solution towards balance. It is an entirely natural thing to try, and it fails all three tests at once:

- (R1) the minimisation in (3.1) is still a QP with the *same dense Hessian* Q . A single scalar constraint has been removed and the whole difficulty left in place;
- (R2) θ is one number, so the Lagrangian dual is univariate and a bisection solves it in a handful of evaluations — there is nothing for a bundle method to do;

- **(R3)** worse, when $Q \succ 0$ — which is the typical case for a Gaussian kernel — the minimiser of (3.1) is unique, hence $\psi \in C^1$ and the dual is smooth, so even a nonsmooth method would be the wrong tool.

Relaxing the box $\alpha \leq C\mathbf{1}$ instead fares no better: the relaxation is again a QP in Q , and now with m multipliers but no easier a subproblem.

The reason is structural: in (D) *no constraint is the difficulty*. The difficulty is Q itself, a dense $m \times m$ matrix. So the object to dualise is not a constraint of (D) at all — it is the identity that *created* Q .

3.3 Putting w back: dualising the representer identity

Q appears in (D) only because w was eliminated through the representer identity (2.2). In the standard derivation that identity is used as a *substitution*: a fact about the optimum, exploited to remove a variable. A substitution cannot be dualised — but a constraint can. So we put w back as an independent variable and *impose* the identity instead of using it:

$$\min_{\alpha, w} \quad \frac{1}{2}\|w\|^2 - \mathbf{1}^\top \alpha \quad \text{s.t.} \quad w = \sum_{i=1}^m \alpha_i y_i \phi_i, \quad \alpha \in D. \quad (\text{D}')$$

Substituting the constraint recovers (D) exactly, so (D') and (D) are the same problem. Now dualise *only* the linking identity, with a free multiplier $\lambda \in \mathcal{H}$:

$$L(\alpha, w; \lambda) = \frac{1}{2}\|w\|^2 - \mathbf{1}^\top \alpha + \left\langle \lambda, w - \sum_i \alpha_i y_i \phi_i \right\rangle.$$

The point of doing this is that w and α are now *untied*. They are two descriptions of the same hyperplane, and (D') forces them to agree; relaxing that constraint lets them disagree and charges a price λ for the disagreement, so that each may be optimised in whichever form suits it. The two groups of variables therefore separate. Minimising over $w \in \mathcal{H}$ is unconstrained and gives $w(\lambda) = -\lambda$ with value $-\frac{1}{2}\|\lambda\|^2$; what remains is *linear* in α over D , because α now appears only linearly. The matrix Q never forms: it existed only because w and α were the same object, and they no longer are. Hence

$$\boxed{\psi(\lambda) = -\frac{1}{2}\|\lambda\|^2 + \min_{\alpha \in D} c(\lambda)^\top \alpha, \quad c_i(\lambda) = -(1 + y_i \langle \lambda, \phi_i \rangle),} \quad (3.2)$$

and the Lagrangian dual of (D') is $\max_{\lambda \in \mathcal{H}} \psi(\lambda)$.

In the vocabulary of partial Lagrangian relaxation, the representer identity is the *complicating* constraint — it is what couples w to α and so what creates the dense Q — while $\alpha \in D$ are the *easy* constraints, kept in the relaxed problem because they are exactly what makes it a knapsack. The usual warning that $\psi(\lambda)$ may be $-\infty$ for some λ , and that one then has to neutralise the offending directions of zero curvature, does not apply here: D is compact, so the relaxed problem always has a finite optimum and ψ is finite on all of $\mathcal{H}$.

3.4 The relaxed problem is a continuous knapsack

Of the two pieces the relaxed problem splits into, the one in w was disposed of in closed form above. The one that is actually solved at every oracle call is the piece in α :

$$\min c^\top \alpha \quad \text{s.t.} \quad y^\top \alpha = 0, \quad 0 \leq \alpha \leq C\mathbf{1}, \quad (\text{KP})$$

a linear objective, box bounds, and *exactly one* further linear constraint. That is the defining shape of the classical *continuous knapsack* problem — continuous, or fractional, because each α_i may take any value in $[0, C]$, where the 0/1 knapsack would have to take an item whole. It is worth a name because it is one of the problems that can be solved exactly by sorting, with no linear programming solver involved. Relaxing that one equality with a scalar θ makes the box minimiser explicit,

$$\alpha_i(\theta) = \begin{cases} C, & c_i - \theta y_i < 0, \\ 0, & c_i - \theta y_i > 0, \\ \text{any value in } [0, C], & c_i - \theta y_i = 0, \end{cases}$$

and $h(\theta) := y^\top \alpha(\theta)$ is non-decreasing, so a θ^* with $h(\theta^*) = 0$ exists and is found by sorting the m breakpoints $y_i c_i$ and scanning (or by median search in $O(m)$ expected time). Splitting the tied components to make the equality hold exactly produces an α that minimises the Lagrangian of (KP) *and* is feasible, hence optimal. Forming c costs one kernel product; the rest costs $O(m \log m)$. This settles (R1): a dense QP has been replaced by a sort. The scalar θ introduced here to solve (KP) is not merely a device: Section 3.6 shows that it is the SVM bias b .

The same structure — relax a single equality, obtain a separable problem whose solution is piecewise linear in the multiplier with one breakpoint per variable — appears as a worked exercise in the course material for the projection onto a hyperplane, and (KP) is its bounded-variable analogue.

3.5 Properties of the dual function

Proposition 3.1 (finiteness). *D is a non-empty compact polytope, so the minimum in (3.2) is attained and finite for every λ . Hence $\psi : \mathcal{H} \rightarrow \mathbb{R}$ is finite everywhere, concave and upper semicontinuous.*

This is unusually convenient: no feasibility (“infinite”) cuts are ever needed, and every oracle call returns a valid lower bound on the optimal value.

Proposition 3.2 (structure and non-differentiability). *Write*

$$\psi = \psi_{\text{sm}} + \psi_{\text{pl}}, \quad \psi_{\text{sm}}(\lambda) = -\frac{1}{2}\|\lambda\|^2, \quad \psi_{\text{pl}}(\lambda) = \min_{\alpha \in D} c(\lambda)^\top \alpha.$$

Then ψ_{sm} is smooth and known in closed form, and ψ_{pl} is a minimum of finitely many affine functions of λ (one per vertex of D), hence concave polyhedral. ψ fails to be differentiable exactly where (KP) has multiple optima, i.e. where some reduced cost vanishes, $c_i - \theta^ y_i = 0$.*

Unwinding that condition with $w = -\lambda$ gives $y_i \langle w, \phi_i \rangle + \theta^* y_i = 1$: the kinks of ψ sit *exactly* on the support-vector condition $y_i f(x_i) = 1$ of (2.3). Since any non-degenerate SVM has free support vectors, the maximiser of ψ is a point of non-differentiability. This settles (R3): the non-smoothness is not incidental, it is the solution.

Proposition 3.3 (no duality gap). *(D') is convex, its coupling constraint is affine and D is compact; therefore $\max_\lambda \psi(\lambda) = v(D') = v(D)$, and the maximiser is $\lambda^* = -w^*$.*

The multiplier of the relaxed identity is thus, up to sign, the primal weight vector itself, and the bundle method’s task can be read as finding the price at which w and α agree again. A supergradient of ψ at λ is the violation of the relaxed constraint,

$$g(\lambda) = w(\lambda) - \sum_i \alpha_i(\lambda) y_i \phi_i = -\lambda - \sum_i \alpha_i(\lambda) y_i \phi_i, \quad (3.3)$$

available at no extra cost from the oracle. (R2) holds as well: λ ranges over $\mathcal{H}$, of dimension n for the linear kernel and m after the reduction of Section 4.4 in general.

3.6 What this dual actually is

It is worth identifying $\max_\lambda \psi$ in closed form. Substituting $w = -\lambda$ in (3.2),

$$\max_\lambda \psi(\lambda) \iff -\min_w \left\{ \frac{1}{2}\|w\|^2 + \max_{\alpha \in D} \sum_i \alpha_i (1 - y_i \langle w, \phi_i \rangle) \right\},$$

and dualising the single equality inside the inner maximisation with a scalar θ gives, by LP duality,

$$\max_{\alpha \in D} \sum_i \alpha_i (1 - y_i g_i) = \min_{\theta} C \sum_i [1 - y_i (g_i + \theta)]^+, \quad g_i := \langle w, \phi_i \rangle. \quad (3.4)$$

So the Lagrangian dual of (D') is the hinge-loss primal (2.1), and *the multiplier θ of the knapsack equality is the SVM bias b* — so b falls out of the oracle for free. That the dual of the dual is the primal is of course biconjugacy; the next section says what is nevertheless gained.

3.7 Relation to the direct nonsmooth formulation

The SVM appears in the course material on nondifferentiable unconstrained optimization as the motivating example, in the form

$$\min_{w,b} \|w\|^2 + C \sum_{i \in I} \max\{1 - y_i(\langle w, x_i \rangle - b), 0\},$$

whose nondifferentiability is precisely what motivates bundle methods. Since Section 3.6 shows that $\max_{\lambda} \psi$ is that same function, we must say clearly what dualising the representer identity adds over simply running a bundle method on it — which is also what the published cutting-plane methods for regularised risk minimisation do (Joachims 2006; Teo et al. 2010; Franc & Sonnenburg 2008).

The answer is *not* that the bias makes a direct approach impossible: $f(w, b)$ is convex in (w, b) jointly and a subgradient in both variables is immediate, so one can perfectly well put b in the bundle. The answer is that our relaxation minimises over b *exactly*, at every oracle call, instead of approximating the b direction with cutting planes. The knapsack multiplier θ of (3.4) *is* that exact partial minimisation, and the sort of Section 3.4 is how it is computed. Equivalently: we bundle $g(w) = \min_b f(w, b)$ rather than $f(w, b)$, and g is the better-conditioned object because one of its directions has been eliminated in closed form. The measured effect is reported in Section 7.2.

We claim exactly this much: the family of cuts and the function being optimised are the same as in the direct approach, and what relaxing the representer identity adds is (i) the exact elimination of b at every oracle call, (ii) the SVM dual variables through the aggregated solution of Section 4.3, and (iii) a two-sided bound on $v(D)$ rather than a bound on one's own objective only.

Chapter 4

The Bundle Method

4.1 The cutting-plane model and the bundle

Nothing about ψ is available in closed form. All an algorithm may do is call the oracle of Chapter 3 at a point λ and receive two things: the value $\psi(\lambda)$ and one supergradient $g(\lambda)$ from (3.3). A bundle method is built on the observation that this is more information than it looks.

Because ψ is concave, a single oracle call yields a bound that is valid *everywhere*, not just near λ :

$$\psi(\mu) \leq \ell_\lambda(\mu) := \psi(\lambda) + \langle g(\lambda), \mu - \lambda \rangle \quad \forall \mu \in \mathcal{H}.$$

Collecting these as the algorithm proceeds gives the *bundle* $\mathcal{B} = \{(\lambda^h, \psi^h, g^h)\}$, and their pointwise minimum

$$\psi_{\mathcal{B}}(\mu) = \min\{\ell_{\lambda^h}(\mu) : (\lambda^h, \psi^h, g^h) \in \mathcal{B}\} \geq \psi(\mu)$$

is the *cutting-plane model* of ψ : an upper approximation that improves wherever the oracle has been called. Maximising the model in place of ψ is the *master problem*, and unlike ψ it can actually be solved, being a small linear or quadratic program.

Taken bare this does not work: where few cuts have been placed the model is a gross over-estimate, so its maximiser can be arbitrarily far from anything yet explored, and the iterates oscillate. The fix is to stabilise. One keeps a *stability centre* $\bar{\lambda}$ — in practice the best point found so far — and penalises displacement from it,

$$\max_{\mu} \psi_{\mathcal{B}}(\mu) - \frac{1}{2t} \|\mu - \bar{\lambda}\|^2,$$

where the *proximal parameter* $t > 0$ expresses how far from $\bar{\lambda}$ the model is trusted. (The course slides write this with a parameter $\mu = 1/t$ multiplying

rather than dividing; the two conventions are reciprocal.) The stabilised master is always bounded above, so its solution is always well defined, which the unstabilised one need not be.

Whether the model deserved that trust is then tested. If the true ψ improves by at least a fixed fraction $\kappa \in (0, 1)$ of the improvement the model predicted, the model is behaving and the centre moves to the new point — a *serious step*, after which t may be increased to allow longer steps. Otherwise the centre stays and only the new cut is kept — a *null step*, after which t is decreased. Either way the bundle has grown where the model was wrong, so a null step is progress too.

4.2 The structured master problem

We depart from the generic scheme of Section 4.1 in one respect. Writing $\psi = \psi_{\text{sm}} + \psi_{\text{pl}}$ as in Chapter 3, the smooth part $\psi_{\text{sm}}(\lambda) = -\frac{1}{2}\|\lambda\|^2$ is known exactly, so there is nothing to be gained by approximating it with cuts and something to be lost, since a piecewise-linear model of a curved function is poor near its maximiser. We therefore linearise only ψ_{pl} and carry ψ_{sm} into the master unchanged — a *generalized*, or structured, bundle method. Each oracle call returns some $\alpha^j \in D$, which gives the affine upper model

$$\psi_{\text{pl}}(\lambda) \leq c(\lambda)^\top \alpha^j = -\mathbf{1}^\top \alpha^j - \langle \lambda, \sum_i \alpha_i^j y_i \phi_i \rangle =: -a_j - \langle \lambda, b_j \rangle.$$

With a stability centre $\bar{\lambda}$ and proximal parameter $t > 0$ the master is

$$\max_{\lambda \in \mathcal{H}} -\frac{1}{2}\|\lambda\|^2 + \min_{j=1,\dots,J} \{-a_j - \langle \lambda, b_j \rangle\} - \frac{1}{2t}\|\lambda - \bar{\lambda}\|^2. \quad (4.1)$$

Exchanging min and max and eliminating λ gives its dual, a QP over the unit simplex Δ^J :

$$\min_{\vartheta \in \Delta^J} \frac{1}{2\rho} \left\| \frac{\bar{\lambda}}{t} - \sum_j \vartheta_j b_j \right\|^2 - \sum_j \vartheta_j a_j, \quad \rho := 1 + \frac{1}{t}, \quad (4.2)$$

after which $\lambda^{\text{new}} = (\bar{\lambda}/t - \sum_j \vartheta_j b_j)/\rho$. Only the $J \times J$ Gram matrix $G_{jl} = \langle b_j, b_l \rangle$ and the vector $p_j = \langle b_j, \bar{\lambda} \rangle$ are needed; both are maintained incrementally, so a master costs $O(J^2)$ plus one kernel product per new cut. We solve it with an off-the-shelf QP solver, which the project statement allows.

Collecting the pieces, (A1) is the following procedure.

$\bar{\lambda} \leftarrow 0$; call the oracle; $\mathcal{B} \leftarrow \{\alpha^1\}$
repeat
 solve the master (4.2) for ϑ ; set λ^{new} and $\hat{\alpha} = \sum_j \vartheta_j \alpha^j$

```

lb  $\leftarrow$  max{lb,  $\psi(\lambda^{\text{new}})$ };  ub  $\leftarrow$   $\frac{1}{2}\hat{\alpha}^\top Q\hat{\alpha} - \mathbf{1}^\top \hat{\alpha}$ 
if ub - lb  $\leq \varepsilon$  then stop
if  $\psi(\lambda^{\text{new}}) \geq \psi(\bar{\lambda}) + \kappa[\text{model}(\lambda^{\text{new}}) - \psi(\bar{\lambda})]$ 
    then  $\bar{\lambda} \leftarrow \lambda^{\text{new}}$ ;  $t \leftarrow \text{larger}$       (serious step)
    else  $t \leftarrow \text{smaller}$       (null step)
 $\mathcal{B} \leftarrow \mathcal{B} \cup \{\alpha^{\text{new}}\}$ , pruning if it has grown too large
until done

```

Note the stopping rule: rather than the usual test on the length of the step, we can stop on the *certified gap* of Section 4.3, which brackets $v(D)$ from both sides and is available at no extra cost.

4.3 Primal recovery and Dantzig–Wolfe

Let $\hat{\alpha} := \sum_j \vartheta_j \alpha^j$ be the *aggregated solution* built from the master problem’s dual multipliers. Since $\vartheta \in \Delta^J$ and D is convex, $\hat{\alpha} \in D$ — it is a *feasible* solution of (D) at every iteration. Moreover $\sum_j \vartheta_j b_j = \Phi^\top Y \hat{\alpha}$ and $\sum_j \vartheta_j a_j = \mathbf{1}^\top \hat{\alpha}$, so for $t \rightarrow \infty$ (4.2) becomes

$$\min_{\vartheta \in \Delta^J} \frac{1}{2} \hat{\alpha}^\top Q \hat{\alpha} - \mathbf{1}^\top \hat{\alpha},$$

i.e. *the SVM dual (D) restricted to $\text{conv}\{\alpha^1, \dots, \alpha^J\} \subseteq D$* . The method is therefore exactly Dantzig–Wolfe decomposition of (D) with (KP) as pricing problem, and bundle, cutting planes on (2.1) and column generation on (D) are three views of one algorithm. Two practical consequences:

- the master returns the primal solution $\hat{\alpha} \rightarrow \alpha^*$, so nothing extra is needed to recover the classifier;
- $\psi(\lambda) \leq v(D) \leq \frac{1}{2}\hat{\alpha}^\top Q\hat{\alpha} - \mathbf{1}^\top \hat{\alpha}$ at *every* iteration, a **certified gap** — which decomposition algorithms for the SVM such as SMO do not provide.

4.4 Kernelised form

For a general kernel $\mathcal{H}$ may be infinite-dimensional, so write $\lambda = \sum_j \beta_j \phi_j$. This is without loss of generality: a component of λ orthogonal to $\text{span}\{\phi_i\}$ leaves $c(\lambda)$ unchanged and only increases $\|\lambda\|^2$, so it is zero at the optimum. Then $\langle \lambda, \phi_i \rangle = (K\beta)_i$ and $\|\lambda\|^2 = \beta^\top K\beta$, and one oracle call costs one matrix–vector product with K plus a sort. Note that the correct stabilising

term is the *RKHS* norm $\|\lambda - \bar{\lambda}\|^2 = (\beta - \bar{\beta})^\top K(\beta - \bar{\beta})$ rather than the Euclidean $\|\beta - \bar{\beta}\|^2$; it is the K -metric that makes (4.2) come out in the form above.

4.5 Cost, and how many cuts are needed

Per iteration: one oracle call ($O(m^2)$ for the kernel product, $O(m \log m)$ for the sort) plus one master ($O(J^2)$ in the QP, $O(Jm)$ to update G). The interesting question is how large J must grow. The oracle returns essentially *vertices* of D , that is $0/C$ vectors, and $\hat{\alpha}$ is a convex combination of them: a component of α^* sitting at a bound is produced by a single vertex, but every *free* support vector $0 < \alpha_i^* < C$ must be produced by mixing vertices that disagree on i . We therefore expect, and measure in Section 7.3, that the number of cuts the method needs is governed by the number of free support vectors, and that truncating the bundle below that number makes the gap stall — the classical Dantzig–Wolfe tailing-off.

Chapter 5

The General-Purpose Solver

For (A2) the dual (D) is handed as a dense convex QP to an off-the-shelf interior-point solver. Mapping it onto the generic QP interface $\min\{\frac{1}{2}z^\top Hz + f^\top z : A_{\text{eq}}z = b_{\text{eq}}, l \leq z \leq u\}$ requires only

$$H = Q, \quad f = -\mathbf{1}, \quad A_{\text{eq}} = y^\top, \quad b_{\text{eq}} = 0, \quad l = 0, \quad u = C\mathbf{1},$$

and the bias is read off as the multiplier of the single equality row, exactly as in Section 2.3. This is the reference both for solution quality — we verify that (A1) and (A2) return the same α^* , b and test accuracy — and for running time.

Chapter 6

Multiclass SVM

6.1 One-vs-Rest approach

The SVM is a binary classifier. With q classes and labels $c_i \in \{1, \dots, q\}$, One-vs-Rest solves q independent binary problems: for class k set

$$y_i^k = \begin{cases} +1, & c_i = k \\ -1, & c_i \neq k \end{cases}$$

and solve (P)/(D) with $y = y^k$, obtaining $f^k(x) = \sum_i \alpha_i^k y_i^k \kappa(x_i, x) + b^k$. A new point is assigned to $\arg \max_k f^k(x)$. It constructs q models, against $q(q-1)/2$ for One-vs-One.

Three observations matter for what follows:

1. The q problems share *the same* kernel matrix K ; they differ only through the label vector, since $Q^k = Y^k K Y^k$ and D^k depends on k only through the single equality $(y^k)^\top \alpha = 0$. Q^k is never formed.
2. The classifiers are trained independently but compared at prediction time on a common scale that nothing in the training enforces; this produces the familiar One-vs-Rest pathologies, ties ($f^k(x) > 0$ for two k) and rejections ($f^k(x) < 0$ for all k).
3. Each binary problem is unbalanced, roughly $1 : (q-1)$.

6.2 Does solving one problem help the others?

The project statement asks this explicitly. The q problems differ only through y^k , so there are three candidate mechanisms, and they are not equally useful.

The kernel matrix. K is computed once and shared; $Q^k = Y^k K Y^k$ is never formed, since the oracle only ever needs $K\beta$. Better still, the q oracles can be run in lockstep so that their q matrix–vector products become one BLAS-3 product $K[\beta^1 \cdots \beta^q]$. This is a genuine, structural saving of a factor q in the dominant cost.

Warm-starting the multiplier. Since $\lambda^* = -w^*$, warm-starting class $k+1$ at class k 's optimum amounts to assuming the two separating hyperplanes are similar. They are not: the positive class changes completely between the two problems.

Transferring cuts. Any point of D^l generates a valid cut for class l , and the D^k differ only through the equality. Writing P_k, P_l for the two positive blocks and R for the rest, $\alpha \in D^k$ satisfies $\alpha(P_k) = \alpha(P_l) + \alpha(R)$, so $(y^l)^\top \alpha = -2\alpha(R)$: the imbalance is known in closed form and can be repaired in $O(m)$ by moving mass onto P_l , *preserving the support pattern* of the transferred cut. This costs no oracle call and no kernel product.

Section 7.4 reports what each of the three is actually worth.

Chapter 7

Experiments

All results in this chapter are produced by the accompanying code; the correctness checks are in `run_checks.py` and the baseline comparison in `compare_baseline.py`.

7.1 Correctness

- **The oracle.** Over 40 random instances the $O(m \log m)$ continuous knapsack of Section 3.4 matches a general LP solver to $1.4 \cdot 10^{-12}$ in objective value, with the equality satisfied and the bounds respected to the same accuracy.
- **Strong duality.** $\max_{\lambda} \psi = v(D)$ to a relative error below $4 \cdot 10^{-9}$ for linear, Gaussian and polynomial kernels and $C \in \{0.1, 1, 10\}$.
- **Weak duality.** Over 3000 random multipliers, no violation of $\psi(\lambda) \leq v(D)$.
- **Primal recovery.** The aggregated solution matches the QP solver's α^* to $1.8 \cdot 10^{-7}$ relative, and the knapsack multiplier matches its b to $1.4 \cdot 10^{-7}$; separately, (3.4) was checked as an identity over 200 random score vectors with no mismatch.
- **End to end.** One-vs-Rest built on (A1) and on (A2) gives identical test accuracy: IRIS 0.956 linear and 0.978 Gaussian, WINE 1.000 for both kernels.

7.2 Against the direct nonsmooth baseline

To isolate the effect discussed in Section 3.7 we compare against a bundle method applied directly to $f(w, b)$, with b as a bundle variable, keeping everything else identical: the quadratic term treated exactly, a cutting-plane model of the rest, the same proximal term and the same master solver.

dataset	kernel	b in the bundle	b eliminated (ours)
WINE	linear	137	29
WINE	Gaussian	400 (gap $6 \cdot 10^{-5}$)	133 (gap $1.1 \cdot 10^{-8}$)
IRIS	Gaussian	96	24
CANCER	linear	220	52

Table 7.1: Iterations to a relative gap of 10^{-8} . The Gaussian WINE baseline had not reached tolerance at the 400-iteration cap.

A consistent factor of four to five, and one instance where the direct method does not reach tolerance at all.

7.3 Bundle size and free support vectors

Section 4.5 predicts that the number of cuts needed is governed by the number of free support vectors. On WINE ($m = 178$) the linear kernel yields 12 free support vectors and the method converges to a gap of $2 \cdot 10^{-8}$ in 26 iterations; the Gaussian kernel with $\gamma = 0.2$ yields 91 free support vectors, and with the bundle capped at 60 cuts the gap stalls at $2 \cdot 10^{-4}$, while raising the cap to 200 restores convergence in 153 iterations. The practical rule is to size the bundle above the expected number of free support vectors, which large C and simple kernels make small.

We also implemented the standard pruning rule — drop a cut whose master multiplier has been zero for 15 consecutive masters — and measured it to be a no-op at these sizes (WINE Gaussian 158 versus 160 iterations, maximum bundle 158 versus 156). This is consistent with the above: nearly every cut is still needed to express $\hat{\alpha}$, so almost none go idle.

7.4 One-vs-Rest transfer

Of the three mechanisms of Section 6.2, only the first is worth anything. Sharing the kernel matrix is worth a factor q in the dominant cost. Warm-starting the multiplier is neutral to harmful: on WINE with a Gaussian kernel

it raised the total oracle-call count by 72%, because the bundle inherits a stability centre far from the new optimum and spends iterations undoing it. Transferring cuts is within noise, between -4% and $+2\%$ oracle calls.

The honest conclusion is that for this decomposition the transferable quantity is the *kernel matrix*, not the dual solution.

Appendix A

The shared-slack-budget model

For completeness we record an alternative model that couples the One-vs-Rest problems, together with a defect we found in it.

A.1 The model

Give each sample a single slack budget B_i to be shared by all q binary classifiers:

$$\begin{aligned} \min_{w, b, \xi} \quad & \sum_{k=1}^q \frac{1}{2} \|w^k\|^2 + C \sum_{i,k} \xi_i^k \\ \text{s.t.} \quad & y_i^k (\langle w^k, \phi_i \rangle + b^k) \geq 1 - \xi_i^k, \quad \xi_i^k \geq 0, \quad \sum_{k=1}^q \xi_i^k \leq B_i. \end{aligned} \tag{M-B}$$

A.2 Its dual

Dualising only the mq margin constraints with $\alpha_i^k \geq 0$ separates the variables. Minimising over w^k gives $w^k = \sum_i \alpha_i^k y_i^k \phi_i$ and $-\frac{1}{2}(\alpha^k)^\top Q^k \alpha^k$; minimising over b^k forces $\sum_i \alpha_i^k y_i^k = 0$; and minimising over ξ splits across i into the tiny LP $\min\{\sum_k (C - \alpha_i^k) \xi_i^k : \xi_i^k \geq 0, \sum_k \xi_i^k \leq B_i\}$, solved by inspection: put the whole budget on the most negative coefficient, or use none of it. Hence

$$\psi(\alpha) = \sum_{i,k} \alpha_i^k - \frac{1}{2} \sum_k (\alpha^k)^\top Q^k \alpha^k - \sum_i B_i \left[\max_k (\alpha_i^k - C) \right]^+, \tag{A.1}$$

on $\{\alpha \geq 0 : \sum_i \alpha_i^k y_i^k = 0 \ \forall k\}$. The budget replaces the box $\alpha \leq C$ of (D) by a penalty, and the dual is non-differentiable where two or more classes tie for a contested budget, or at the kink $\max_k \alpha_i^k = C$ — which is where the One-vs-Rest coupling actually happens.

A.3 The defect, and its repair

With a *hard* budget, (M-B) can be infeasible, and then its Lagrangian dual is unbounded. This is not a corner case: a sample lying where two classifiers both score near zero needs $\xi_i^k \approx 1$ for each of them, so $B_i < 2$ is typically already infeasible. On a three-class Gaussian blob instance we measured the feasibility threshold at exactly $B_{\min} = 2.0$, with (A.1) diverging for every $B < B_{\min}$ and strong duality holding to 10^{-8} for every $B \geq B_{\min}$. A single outlier destroys the training problem.

The repair is to soften the budget: $\sum_k \xi_i^k \leq B_i + z_i$ with $z_i \geq 0$ penalised by Dz_i . The ξ subproblem stays an LP solved by inspection, the model is always feasible, and boundedness of the z term restores a box, $0 \leq \alpha \leq (C+D)\mathbf{1}$, making $\text{dom } \psi$ compact. We verified strong duality for the repaired model to 10^{-8} across a range of B , D , q and m .

A.4 Why it is not the main line

Even repaired, (A.1) retains the quadratic forms Q^k *inside* the dual function, so the bundle's piecewise-linear model has to approximate a curved function and the method degenerates into solving a QP by cutting planes. The relaxation of Chapter 3, by contrast, leaves an LP and keeps its only curved term in closed form in the master. And (M-B) changes the model (M), so any comparison against One-vs-Rest would confound a modelling change with an algorithmic one, whereas Chapter 3 leaves (M) exactly the standard One-vs-Rest SVM and changes only how it is solved.